

Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №5

ШАБЛОНИ «ADAPTER», «BUILDER», «COMMAND», «CHAIN OF RESPONSIBILITY», «PROTOTYPE».

Виконав
студент групи ІА–22:
Артамонов Дмитро
Перевірив:
Мягкий Михайло Юрійович

Київ 2024

Зміст

Короткі теоретичні відомості 3
Реалізація не менше 3-х класів відповідно до обраної теми.....5
Реалізувати один з розглянутих шаблонів за обраною темою 6
Діаграма класів.....8

Тема: шаблони «ADAPTER», «BUILDER», «COMMAND», «CHAIN OF RESPONSIBILITY», «PROTOTYPE».

Мета: ознайомитися з шаблонами проектування «Adapter», «Builder», «Command», «Chain of responsibility», «Prototype» та набути практичних навичок їх застосування. Реалізувати частину функціоналу програми за допомогою одного з розглянутих шаблонів для досягнення конкретних функціональних можливостей та забезпечення ефективної взаємодії між класами.

Хід роботи

..15 E-mail клієнт (singleton, builder, decorator, template method, interpreter, SOA)

Поштовий клієнт повинен нагадувати функціонал поштових програм Mozilla Thunderbird, The Bat і т.д. Він повинен сприймати і коректно обробляти

---

pop3/smtp/imap протоколи, мати функції автонастройки основних поштових провайдерів для України (gmail, ukr.net, i.ua), розділяти повідомлення на папки/категорії/важливість, зберігати чернетки незавершених повідомлень, прикріплювати і обробляти прикріплені файли.

Короткі теоретичні відомості

Шаблони проектування є важливими інструментами для вирішення повторюваних задач у розробці програмного забезпечення. У лабораторній роботі розглядаються п’ять ключових шаблонів:

1. **Adapter (Адаптер)**
   Використовується для узгодження двох несумісних інтерфейсів. Наприклад, адаптер для різних форматів даних. Реалізація зазвичай передбачає створення класу, який перетворює виклики одного інтерфейсу в інший.
2. **Builder (Будівельник)**
   Застосовується для створення складних об’єктів шляхом розділення

процесу на окремі кроки. Це дозволяє легко генерувати різні представлення одного і того ж об'єкта без зміни його структури.

3. **Command (Команда)**
   Інкапсулює дію або виклик методу як об’єкт. Це дозволяє реалізувати відкладене виконання, скасування чи повторення команд, як у системах управління задачами.
4. **Chain of Responsibility (Ланцюжок відповідальності)**
   Організовує обробників у ланцюжок, де кожен з них вирішує, чи обробляти запит, чи передати його далі. Наприклад, перевірка доступу до ресурсу на основі різних правил.
5. **Prototype (Прототип)**
   Застосовується для створення нових об'єктів шляхом копіювання існуючих. Це спрощує створення об'єктів, зменшуючи час і ресурси на їх ініціалізацію.

Анти-шаблони — це помилки проектування, яких слід уникати. Вони виникають через недоліки в аналізі чи реалізації.

- **Smoke and Mirrors**: презентація нереалізованих функцій для створення хибного враження про прогрес.
- **Scope Creep**: постійне додавання нових функцій без чітких меж проекту, що ускладнює його виконання.
- **Vendor Lock-in**: створення системи, яка занадто залежить від одного постачальника технологій, що обмежує її адаптивність.

Ці теоретичні концепції сприяють побудові більш якісних, стійких і зрозумілих програмних рішень.

Реалізація не менше 3-х класів відповідно до обраної теми

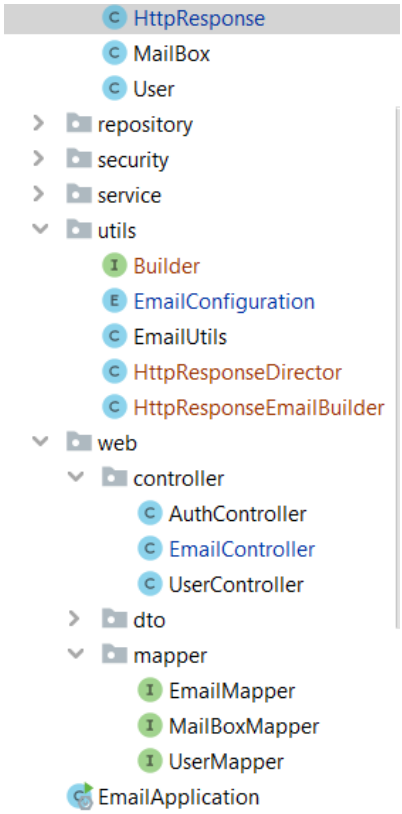


Рисунок 1. Структура реалізованих класів проекту.

Опис класів

1. Інтерфейс Builder визначає метод build(), який відповідає за створення об'єкта типу HttpResponse. Його призначення – стандартизувати та спростити процес побудови HTTP-відповідей із різними параметрами, забезпечуючи гнучкість і розширюваність у системі;
2. Клас HttpResponseEmailBuilder призначений для створення HTTP-відповіді, яка повідомляє про успішну відправку електронного листа. Реалізуючи інтерфейс `Builder`, він інкапсулює логіку формування об'єкта `HttpResponse` з визначеними параметрами, такими як код статусу, повідомлення, шлях та часовий штамп. Цей клас використовується для стандартизації та спрощення процесу створення відповідей у контексті операцій з електронними листами.
3. Клас HttpResponseDirector забезпечує координацію процесу створення об'єкта HttpResponse за допомогою патерну Builder. Його метод buildHttpResponse(Builder builder) отримує реалізацію інтерфейсу Builder та викликає метод build() для створення кінцевого об'єкта.

4. Клас HttpResponse є продуктом, який створюється за допомогою патерну Builder. Він слугує моделлю для представлення HTTP-відповідей у системі, забезпечуючи зручний спосіб зберігання інформації про статус запиту, повідомлення, шлях, метод запиту та додаткові дані.

Реалізувати один з розглянутих шаблонів за обраною темою
Опис шаблону

Було реалізовано наступні компоненти шаблону «Будівельник»:

1. Інтерфейс Builder:

- Визначає метод build(), який має реалізувати кожен конкретний будівельник (Builder).
- Гарантує, що будь-який клас, який реалізує цей інтерфейс, матиме однакову структуру створення об'єкта HttpResponse.

2. Клас HttpResponseEmailBuilder:

- Є конкретною реалізацією інтерфейсу Builder.
- Використовується для створення специфічного об'єкта HttpResponse, який представляє успішну відправку електронного листа.
- Реалізує метод build() для повернення налаштованого HttpResponse з параметрами, такими як HttpStatus.OK, код 200, повідомлення про успіх і шлях /api/v1/emails.

3. Клас HttpResponseDirector:

- Реалізує роль директора в шаблоні Builder.
- Відповідає за контроль процесу створення об'єкта HttpResponse шляхом виклику методу build() у переданого Builder.
- Забезпечує клієнтам зручний спосіб отримання об'єкта без необхідності взаємодії з конкретними будівельниками.

4. Клас HttpResponse

- Є продуктом, що створюється за допомогою шаблону Builder.
- Представляє HTTP-відповідь з різноманітними атрибутами, такими як код статусу, повідомлення, шлях, метод запиту тощо.

- Реалізує статичний вкладений клас HttpResponseBuilder, який інкапсулює логіку побудови об'єкта HttpResponse.
- Методи HttpResponseBuilder дозволяють послідовно налаштовувати властивості об'єкта.
- Метод build() створює і повертає повністю налаштований об'єкт HttpResponse.

Проблему, яку допоміг вирішити шаблон Будівельник.

Використання патерну Builder у проекті допомогло вирішити проблему створення складних об’єктів HttpResponse, які мають багато параметрів. Завдяки цьому підходу вдалося спростити процес їх побудови, забезпечити гнучкість і зручність налаштування для різних сценаріїв, уникнувши громіздких конструкторів і значно підвищивши читабельність коду. Це також спростило підтримку та розширення функціональності класу в майбутньому.

Переваги вкиористання шаблону Будівельник

- **Ізоляція процесу створення об'єкта**: Клієнт не потребує знань про структуру чи порядок ініціалізації полів HttpResponse.
- **Гнучкість**: Можна створювати різні конфігурації об'єкта HttpResponse (наприклад, інші конкретні будівельники можуть реалізовувати інтерфейс Builder для створення інших варіантів HTTP-відповідей).
- **Використання принципу розділення обов’язків (SRP):** Кожен клас виконує свою чітко визначену роль, що підвищує загальну структурованість проекту.
- **Масштабованість:** Нові поля чи параметри можна легко додавати, оновлюючи лише будівельник, що дозволяє розширювати функціональність без значних змін у коді.

Діаграма класів

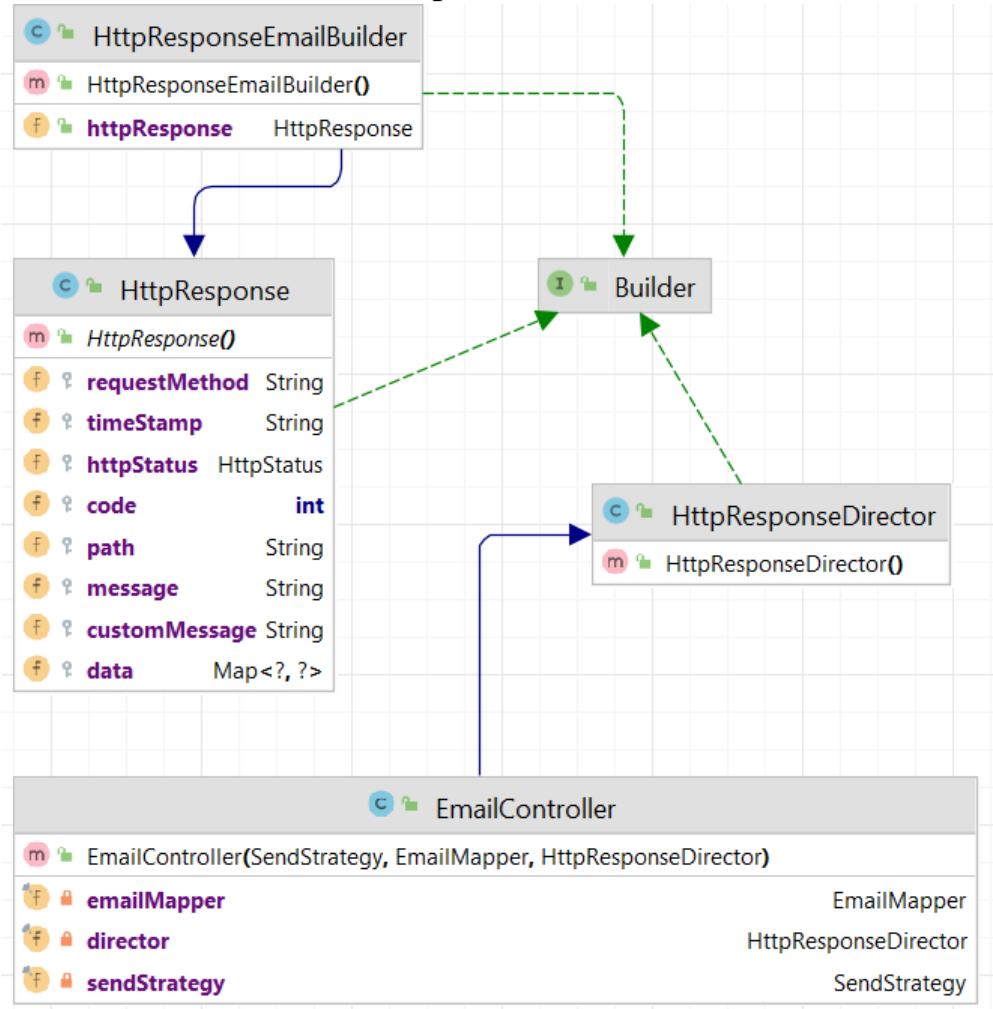


Рисунок 2. Діаграма класів для патерну «Builder».

Код за посиланням: https://github.com/qjsoq/Email-API

Висновок: У даній лабораторній роботі я реалізував патерн «Builder», що спростило створення об'єктів із багатьма параметрами та підвищило гнучкість коду. Завдяки цьому патерну процес ініціалізації об'єктів став зрозумілим і модульним, що полегшило тестування, розширення та підтримку системи.